# Understanding JTAG

A lot of debuggers connect to chips through JTAG, using tools such as OpenOCD. Let’s try to see and decode how this all works.

## Introduction

JTAG began as a way to test connections on assembled PCBs. As surface-mounted chips and dense, multilayered PCB became increasingly common Physical probing of the chips was almost impossible now. So boundary scan offered a way to observe and control signals at chip pins without touching each connection with a probe.

The first IEEE 1149.1 (JTAG) standard came up in 1990 and a lot of improvements have been continuing on.

The Test Access Port (TAP) has four required signals: Test Clock (TCK), Test Mode Select (TMS), Test Data Input (TDI), and Test Data Output (TDO). Test Reset (TRST) is an optional fifth signal. TMS and TDI are sampled on the rising edge of TCK. TDO changes on the falling edge of TCK. Asserting TRST low asynchronously resets the TAP and its test logic **but not** the chip’s normal system logic.

The standard requires undriven TMS and TDI inputs to behave as logic 1; a pull-up resistor is generally connected to implement this as you will understand later as to why.

On a board with several JTAG-capable chips, TCK and TMS can be shared, while TDO of one chip connects to TDI of the next. That forms one serial scan chain instead of needing a separate data connection to every chip.

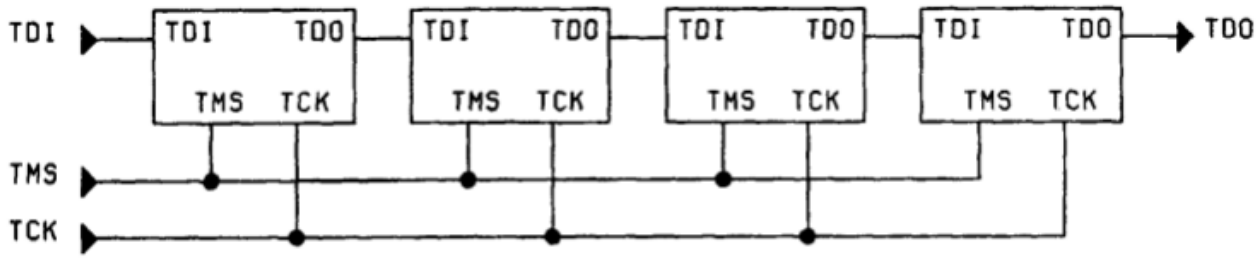



**Figure 4-1—Serial connection using one TMS signal**

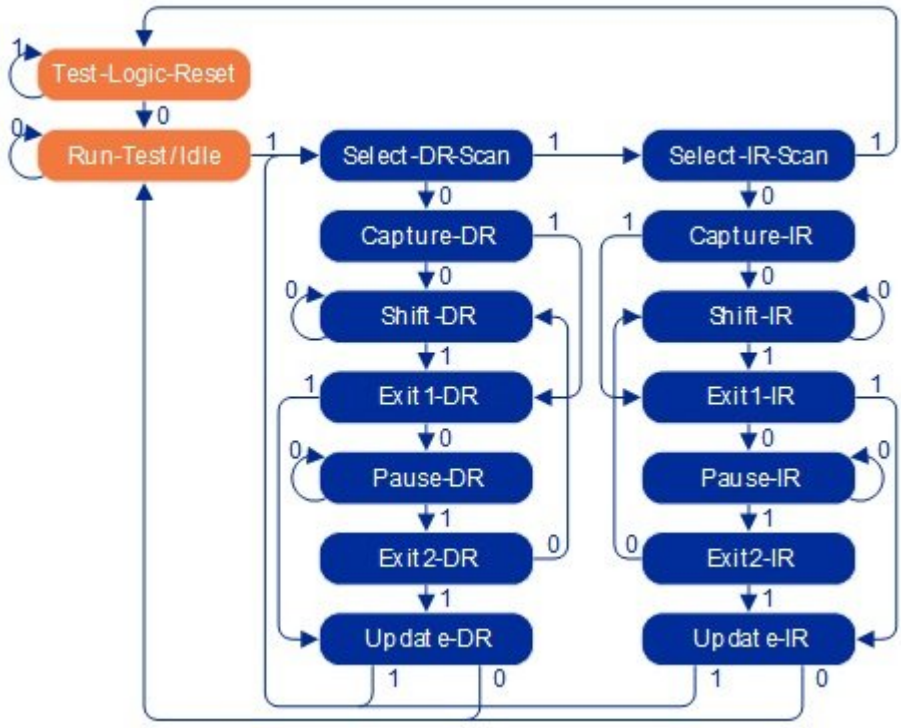
*Serial connection using one TMS signal.*

Each chip on that chain has a TAP controller: a 16-state machine that chooses whether we access the Instruction Register (IR) or a selected Data Register (DR). The value of TMS at each rising TCK edge determines the next state. An instruction selects a test operation, a data-register path, or both.

On reset most of the chips will start as Test-Logic-Reset state and would have **IDCODE** instruction or **BYPASS** latched on which OpenOCD depends on to find the ID of the chip. If you notice carefully continously holding TMS at 1 for at least five rising TCK cycles takes the controller to Test-Logic-Reset from any state. Now it explain the undriven-high requirement for TMS: an open TMS connection should not leave the test logic in a disruptive mode.

The IR length and the selected DR length depend on the chip. For JTAG confirming component, the all-1 IR code selects BYPASS and select its one-bit DR path. If an undriven TDI behaves as 1, a complete IR scan of ones therefore selects BYPASS instruction where, each rising TCK edge shifts one bit in through TDI and moves one bit toward TDO bypassing chip completely. This explains the undriven-high requirement for TDI line.

The TDI and TDO lines are only considered valid in Shift-DR and Shift-IR stage. The old instruction remains active till Update-IR state where it makes a new instruction active. Update-DR state transfers shifted-in data to a register's output latch.




*TAP controller state transitions*

## PCB Testing How?

Chips would have boundary-scan cells near its system pins form a shift-register path around the chip. During normal operation, signals pass through these cells to the on-chip logic. Boundary-scan instructions when selected can instead sample pin values or use the cells to control outputs.

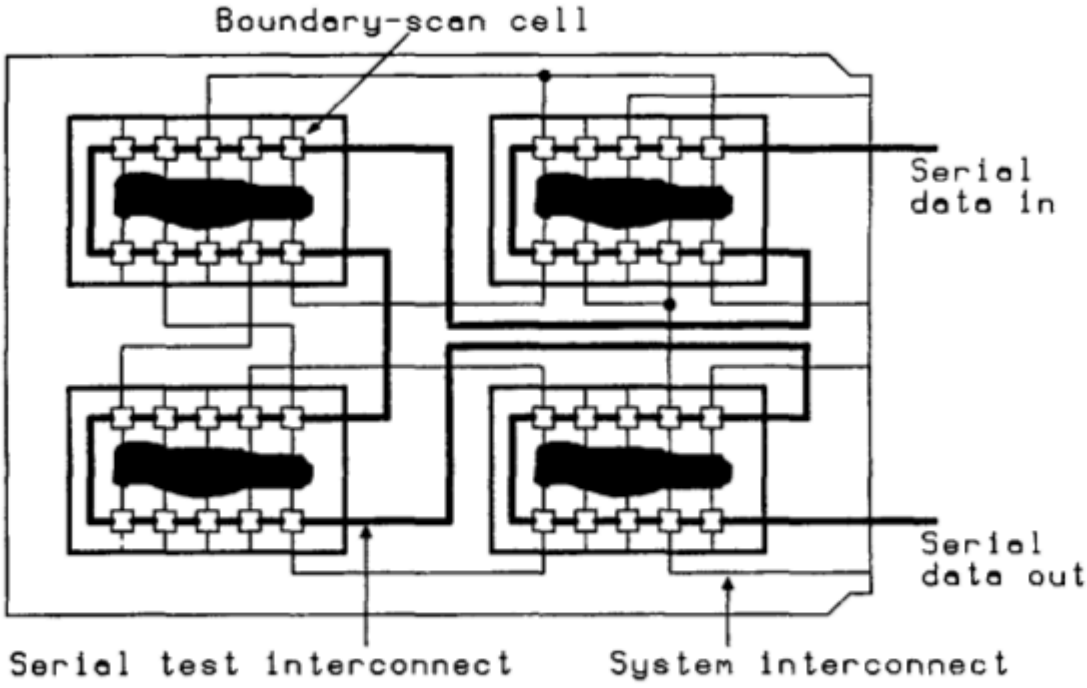



**Figure 1-2—A boundary-scannable board design**

*a board-level view with chips containing boundary scan cell.*

Instructions related to this would be:

1. **SAMPLE** captures a snapshot of signals at the boundary without interrupting normal operation. Capture-DR takes the snapshot; Shift-DR lets us read it through TDO.
2. **PRELOAD** shifts a chosen pattern into the boundary-scan register, then transfers it to the cells' output latches at Update-DR. It prepares known values **without driving those test values onto the pins**.
3. **EXTEST** tests external interconnections: output cells apply a prepared test pattern, and input cells capture what arrives. Preloading first avoids exposing unknown output values when EXTEST starts.

Other optional instructions include HIGHZ, CLAMP, INTEST, and RUNBIST. Their support depends on the component. They allow to test internal logic as well.

## Debugging How?

So how do debuggers use JTAG? Arm's Debug Access Port (DAP) has a Debug Port (DP) and one or more Access Ports (APs). A **JTAG-DP** provides a JTAG interface to that DAP; the IR

instructions select debug scan chains rather than just the boundary-scan operations above.

Instructions related to this would be:

1. **DPACC** — select a DR path for DP register accesses.
2. **APACC** — select a DR path for AP register accesses.
3. **ABORT** — select the DR path for an abort operation.
4. **IDCODE** and **BYPASS** — select the identification and one-bit bypass paths.

See the Arm Debug Interface specification in the references for more details.

## Conclusion

JTAG is an incredible protocol, the same TAP scan mechanism which supports both board testing and access to on-chip debug hardware. Although the particular instructions and data registers depend on the device. The manufacturers generally do supply a BSDL description of the test interface.

There would be some ways to break security as well! Just something to explore I guess.

## References

Included some from where I studied about JTAG are :


- IEEE Std 1149.1-2001: Standard Test Access Port and Boundary-Scan Architecture — primary reference for the TAP and boundary-scan behavior in this post.
- Arm Debug Interface v5 Architecture Specification — JTAG-DP and its debug instructions.
- OpenOCD JTAG primer — a practical introduction.
- AMD/Xilinx XC9500/XL/XV JTAG Boundary Scan Interface — an example device-specific reference.
- EEVblog's JTAG tutorial — helped me see the original board-testing use case.
- Wikipedia ; as always start your journey on research with Wiki! More on debugging as well